Tutoría de Fundamentos de Informática

Programación Orientada a Objetos
y Diagramas UML

Cipriano Rivera Escobar

Fundamentos de Informática

Universidad Nacional de Costa Rica

II Ciclo 2026

Miércoles 21 de octubre de 2026

Objetivo de la tutoría

Objetivo

Brindar a los estudiantes la oportunidad de fortalecer sus habilidades en programación y modelado mediante la resolución de ejercicios prácticos, aplicando los conceptos fundamentales del paradigma de Programación Orientada a Objetos y la representación visual de sistemas mediante diagramas UML.

Temas a abordar

1. Paradigma de Programación Orientada a Objetos (POO).
2. Diagramas UML (Unified Modeling Language).

Ejercicio 1: Campeonato de fútbol femenino

Federación de Fútbol Femenino
Análisis de un problema mediante POO

Situación problemática

Una federación de fútbol femenino requiere desarrollar un programa en C++ que permita administrar los equipos participantes en un campeonato.

De cada equipo es necesario conocer su nombre, los puntos obtenidos, la cantidad de partidos ganados, perdidos y empatados, así como los goles anotados y recibidos.

El sistema debe permitir registrar hasta **36 equipos** y administrar la información de los participantes.

Primer paso: Lea el problema e identifique
la información que debe representar el sistema.

¿Qué debe realizar el sistema?

01. Registro
Agregar equipos sin repetir nombres.

02. Consulta
Buscar, obtener y sustituir equipos.

03. Resultados
Registrar partidos y actualizar estadísticas.

04. Clasificación
Ordenar los equipos por puntos.

05. Ganadores
Obtener una colección con los tres primeros lugares.

Registro de resultados

Para registrar un partido, el sistema recibe los nombres de los dos equipos y los goles anotados por cada uno.

Ambos equipos deben existir en el campeonato. Se deben actualizar sus estadísticas y puntos.

VICTORIA	EMPATE	DERROTA
3 puntos	**1 punto**	**0 puntos**

Al finalizar el campeonato

Los equipos deben ordenarse de mayor a menor cantidad de puntos. Además, se debe obtener una colección independiente con los tres primeros lugares.

Parte 1: Análisis y diagrama UML

Antes de comenzar a programar:

1. Identifique las clases necesarias para representar el problema.
2. Determine los atributos y sus tipos de datos.
3. Identifique los constructores y métodos necesarios.
4. Establezca la relación entre las clases.
5. Elabore los diagramas UML indicando visibilidad, atributos, métodos, parámetros y retornos.

Importante: Primero se realiza el análisis y el UML; después se escribe el código.

Parte 2: Implementación en C++

Una vez terminados los diagramas UML, implemente el sistema.

1. Cree los archivos `.h` y `.cpp` de cada clase.
2. Implemente constructores, métodos de acceso, modificación y representación textual.
3. Desarrolle el registro, búsqueda y administración de equipos.
4. Implemente el registro de resultados y actualización de estadísticas.
5. Ordene los equipos por puntos utilizando un algoritmo estudiado.
6. Obtenga los tres primeros lugares y pruebe el programa en `main.cpp`.

Utilice un arreglo fijo de hasta **36 equipos**

Ejercicio 2: Restaurante MiPedido

Restaurante MiPedido
Sistema de pedidos y beneficios para clientes

Situación problemática

El restaurante **MiPedido** logró mantener sus operaciones durante la pandemia mediante el servicio de comida a domicilio.

Debido al crecimiento del negocio, sus propietarios desean desarrollar un sistema en C++ para automatizar los pedidos de sus clientes y ofrecer beneficios según sus compras.

Cada pedido posee un código identificador, corresponde a un tipo de comida e incluye una determinada cantidad de platos y bebidas.

El restaurante necesita calcular precios, aplicar descuentos, generar facturas y administrar los pedidos semanales.

Actividad inicial: Lea el enunciado e identifique qué información debe almacenar el sistema.

Tipos de pedidos y precios

El precio depende del tipo de pedido y de la cantidad de platos y bebidas.

DESAYUNO
D
Cada plato cuesta **2500 colones**.
Las bebidas son **gratis**.

ALMUERZO
A
Cada plato cuesta **3500 colones**.
Cada bebida cuesta **1000 colones**.

CENA
C
Cada plato cuesta **4500 colones**.
Una bebida gratis por plato.

Cena: Las bebidas adicionales a las obsequiadas cuestan **1000 colones cada una**.

Descuentos según la cantidad de platos

El restaurante aplica descuentos según la cantidad de platos incluidos en cada pedido.

Cantidad de platos	**Descuento**
10 platos o más	15 %
De 8 a 9 platos	10 %
De 5 a 7 platos	5 %
Menos de 5 platos	1 %

El porcentaje de descuento depende de la cantidad de platos.
El precio final se calcula **únicamente al generar la factura**.

Facturación de los pedidos

El sistema debe generar una factura sencilla que incluya:

1. Código identificador del pedido.
2. Tipo de pedido realizado.
3. Cantidad de platos y bebidas.
4. Precio total de los platos.
5. Precio total de las bebidas.
6. Porcentaje de descuento aplicado.
7. Precio final con descuento.

Diseño libre: El formato de la factura puede ser creativo, siempre que presente todos los datos solicitados.

Administración de pedidos semanales

MiPedido necesita administrar los pedidos que realiza cada cliente durante una semana.

Un cliente puede realizar pedidos de distintos tipos, pero no puede superar los **50 pedidos semanales**.

El sistema debe permitir:

1. Registrar y administrar los pedidos de una semana.
2. Mostrar todos los pedidos del cliente.
3. Consultar únicamente los pedidos de desayuno.
4. Determinar el monto total de descuentos semanales.
5. Contar los pedidos con descuento superior al 5 %.
6. Identificar el pedido con mayor cantidad de platos.

Sistema de premios para clientes

El restaurante desea entregar un **sticker de premio** a los clientes que cumplan determinadas condiciones.

Condición 1
Tener más de **10 pedidos** con un descuento superior al 5 %.

Condición 2
Acumular más de **10 000 colones** en descuentos semanales.

Ambas condiciones deben cumplirse. El sistema debe conservar y mostrar si el cliente tiene derecho al premio.

Identificación de pedidos recurrentes

El restaurante desea identificar los pedidos que un cliente repite en distintas semanas.

Dos pedidos son **análogos** cuando coinciden en:

1. El tipo de pedido.
2. La cantidad de platos.
3. La cantidad de bebidas.

Requerimiento:
Comparar los pedidos de dos semanas y generar una colección con los pedidos recurrentes.
Las colecciones pueden tener diferentes cantidades de pedidos.

Parte 1: Análisis y diagramas UML

Antes de escribir el código:

1. Identifique las clases necesarias para resolver el problema.
2. Determine los atributos y sus respectivos tipos de datos.
3. Identifique los constructores y métodos básicos.
4. Defina los métodos para calcular descuentos y generar facturas.
5. Determine cómo administrar los pedidos semanales y los premios.
6. Identifique la relación entre las clases.
7. Construya los diagramas UML con atributos, métodos, parámetros, retornos y visibilidad.

Parte 2: Implementación en C++

Después de elaborar los diagramas UML, implemente la solución.

1. Cree los archivos `.h` y `.cpp` para cada clase.
2. Implemente constructores, métodos de acceso y modificación.
3. Desarrolle el cálculo de descuentos, precios y facturas.
4. Implemente la administración de los pedidos semanales.
5. Determine los beneficios y premios de los clientes.
6. Desarrolle la comparación de pedidos entre semanas.

Utilice un arreglo fijo de hasta **50 pedidos**,

Parte 3: Pruebas del programa

En `main.cpp`, realice las siguientes pruebas:

1. Construya al menos **6 pedidos** con el constructor con parámetros.
2. Pruebe los métodos de cálculo y facturación.
3. Cree dos colecciones: `Sem1` y `Sem2`.
4. Registre al menos **3 pedidos** en cada semana.
5. Asegúrese de que ambas semanas compartan al menos **2 pedidos análogos**.
6. Pruebe todos los métodos desarrollados para administrar los pedidos semanales.

Entrega: Diagramas UML, archivos `.h`, `.cpp` y `main.cpp` con las pruebas.

¡Fin de la tutoría!

Gracias por participar

Fundamentos de Informática · II Ciclo 2026